



SAPTHAGIRI NPS
UNIVERSITY
UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

Sapthagiri NPS University

Department of Computer Science and Engineering

Public Grievance Redressal System



Presented by:

Ranganath T G	24SUUBECS1692
Ranjan Kumar H S	24SUUBECS1693
Ranjita Ganesh Naik	24SUUBECS1694

Under the Guidance of

Faculty Name: Shilpa R
Designation: Assistant Professor
Department: Computer Science and Engineering



Agenda

- Introduction
- Problem Definition
- Database Design
- Tables and Relationships
- SQL Queries / CRUD Operations
- Frontend and Output
- Advantages and Limitations
- Conclusion
- Future Scope
- References



Introduction

Project Overview & Necessity

1. Project Topic: GrievTech

GrievTech is an **Grievance Management System** designed to digitize and automate the reporting of public infrastructure issues. The system bridges the communication gap between the general public and municipal authorities.

2. Why is this project useful?

Instant Reporting: Allows citizens to report issues like potholes, water leaks, or power outages in seconds.

Precise Location: Uses **Geotagging** technology to extract exact GPS coordinates from photos, eliminating manual location errors.

Transparency: Provides a real-time tracking system so citizens know exactly when their problem is being worked on.





Stakeholders & Functional Overview

3. Who will use the system?

Citizens: To securely report public issues, upload geotagged photographic evidence, and track the real-time progress of their complaints.

Admin / Government Authorities:

A single centralized portal for officials to:

- View complaints on the **Interactive Map** (Trace exact locations).
- Prioritize "Emergency" cases (highlighted in Red).
- Analyze data using the **Dynamic Graphs** (Weekly/Monthly reports).
- Update the status to keep citizens informed.

4. Overview of the Project

- Web-based system built using Flask and database
- Users can register, login, and submit complaints with images
- Admin can view, update, and resolve complaints
- **Includes advanced features:**
- Chatbot for user help
- Location-based complaint mapping
- Multi-language support
- Feedback and rating system



Problem Definition

Inefficient and non-transparent manual grievance systems make it difficult for citizens to report, track, and resolve public issues effectively.

Existing Challenges in Public Complaint Systems

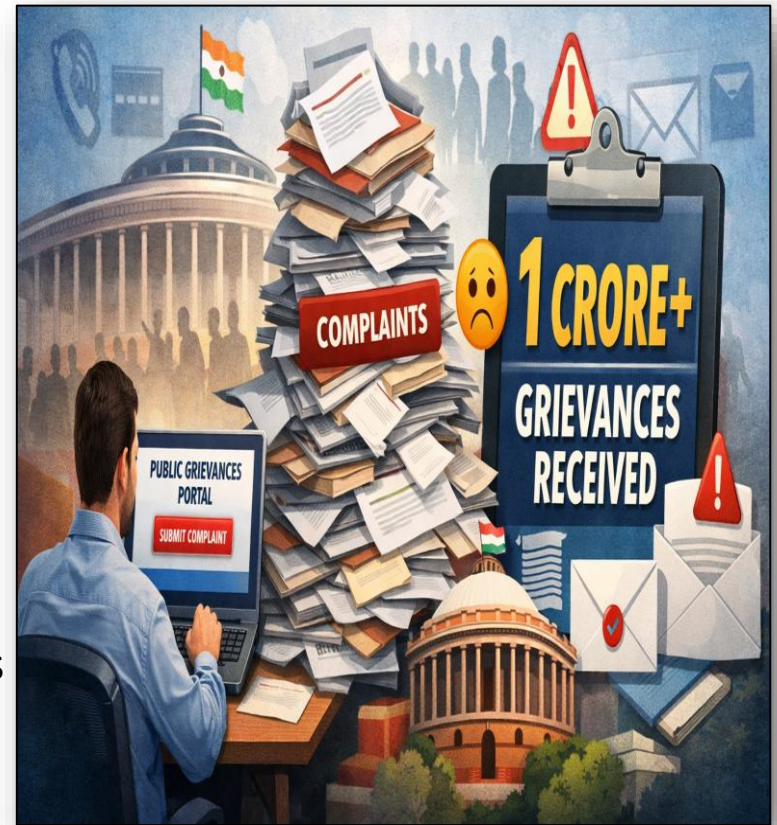
- People face difficulty in reporting issues like water shortage, road damage, garbage overflow, and electricity problems
- Complaints are handled manually or through inefficient systems
- Leads to delays in resolving issues
- Lack of transparency in complaint handling
- Poor communication between citizens and authorities

Limitations of Traditional Approach

- Complaints may get lost, duplicated, or ignored
- No proper tracking system for users
- Users cannot check complaint status easily
- Authorities struggle to manage large numbers of complaints
- Difficult to identify high-priority or critical problem areas

Need for an Intelligent Solution

- Requires a smart and centralized complaint system
- Enables easy submission and tracking of complaints
- Reduces duplicate complaints automatically
- Improves response time and efficiency
- Enhances communication between users and administrators
- Supports advanced features like chatbot assistance
- Provides location-based tracking of complaints
- Offers multilingual support for better accessibility



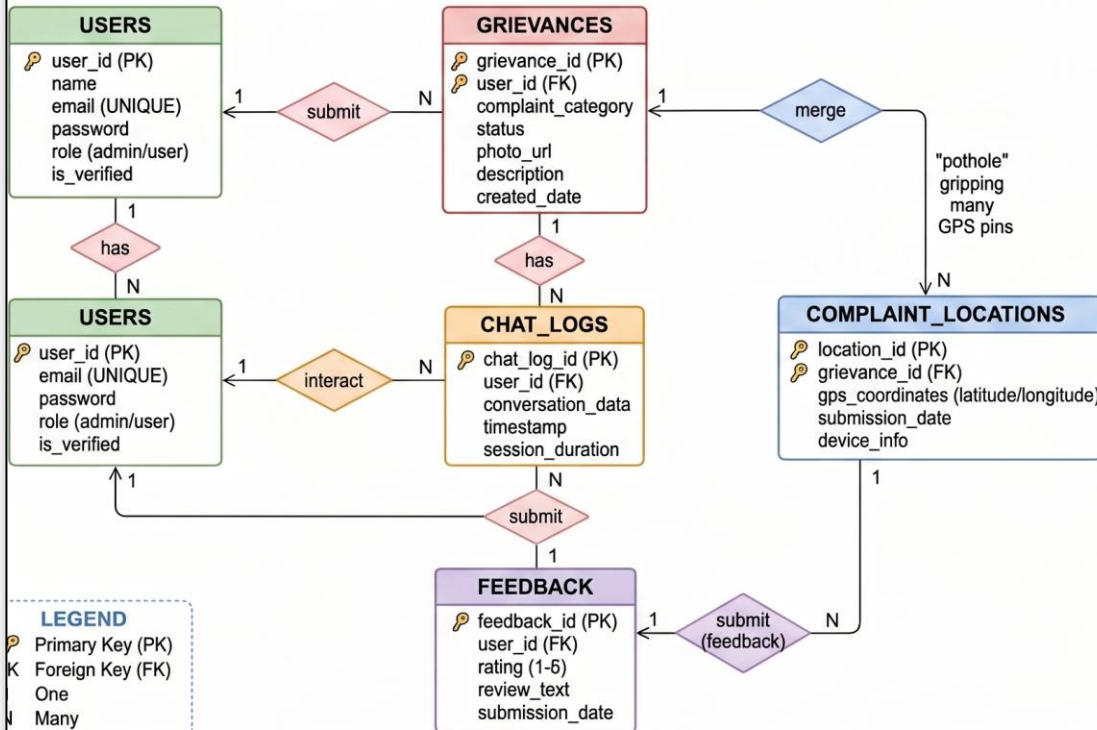
Traditional complaint systems are inefficient and unorganized, making it difficult to manage, prioritize, and resolve public issues effectively.

Database Design

ONLINE GRIEVANCE & CHAT SYSTEM

ER DIAGRAM & RELATIONSHIPS

ER DIAGRAM



RELATIONSHIPS

Relationship	Type	Description
Users → Grievances	1 -----> N (One-to-Many)	One user can submit multiple grievances.
Users → Chat Logs	1 -----> N (One-to-Many)	One user can have multiple interactions with the chatbot.
Users → Feedback	1 -----> N (One-to-Many)	One user can submit multiple feedback rating
Grievances → Locations	1 -----> N (One-to-Many)	One master grievance can hold multiple GPS location pins (from merged duplicates).

KEY POINTS

- Citizens must register/login to submit a grievance or use the chatbot.
- Every grievance is firmly linked to a specific user via a Foreign Key (user_id).
- The system uses Normalization by separating complaint_locations into its own table, allowing one main 'pothole' report to hold multiple GPS pins if multiple people report the same area.
- Referential integrity is maintained (e.g., ON DELETE CASCADE ensures if a grievance is deleted, its map pins are also deleted).

ENTITIES IN THE SYSTEM



USERS
Stores user information, roles (admin/user), and authentication details.



GRIEVANCES
Stores complaint details, categories, status, and the initial uploaded photo.



COMPLAINT_LOCATIONS
Stores precise GPS coordinates for merged duplicate reports.



CHAT_LOGS
Stores the conversation history between the citizen and the AI bot.



FEEDBACK
Stores citizen ratings (1-5) and reviews of the platform.



GRIEVTECH SYSTEM Database Design

ENTITY, ATTRIBUTES & NORMALIZATION (GRIEVTECH SYSTEM)

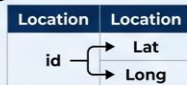
ENTITY	DESCRIPTION	KEY ATTRIBUTES	NOTES
 USERS	Stores information about registered citizens and admins.	- id (PK) - name - email (UNIQUE) - password - role	- Users log in to report issues. - Admins get special dashboard access.
 GRIEVANCES	Stores details of complaints submitted by citizens.	- id (PK) - user_id (FK) - category - latitude / longitude - status	- The core table of the system. - Tracks real-time status and duplicate counts.
 COMPLAINT LOCATIONS	Stores exact coordinates for merged/duplicate reports.	- id (PK) - complaint_id (FK) - user_id (FK) - latitude / longitude	- Powers the Geotagged Heatmap. - Links multiple geographic spots to one main issue.
 CHAT_LOGS	Stores AI chatbot conversation history.	- id (PK) - user_id (FK) - user_message - bot_response	- Secures interaction logs. - Helps admins review common citizen queries.
 FEEDBACK	Stores system ratings and reviews.	- id (PK) - user_id (FK) - rating (1-5) - message	- Enforces constraints (rating must be between 1 and 5). - Linked to the user profile.

DATABASE NORMALIZATION (APPLIED 3NF)

To ensure optimal performance, eliminate data redundancy, and save server storage, the GrievTech database is strictly normalized up to the Third Normal Form (3NF).

1 1NF (First Normal Form)

All tables have a unique Primary Key (id), and every column holds atomic values.



1 1NF (First cond Normal Form)

All tables have a unique Primary Key nment on the Primary Key.



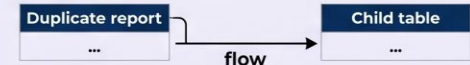
2 2NF (Second Normal Form)

All non-key attributes are fully dependent on the Primary Key.



3 3NF & Intelligent Duplicate Handling

Core optimization. Extracted location data (complaint_locations) from duplicates prevents duplication of heavy data (images/text) in grievances.



The Benefit

Heavy data is saved only once. Duplicates only save new GPS coordinates. Drastically reduces server load and ensures the admin Heatmap renders instantly.



Tables and Relationships

1. Important Tables Used in the Project:

- **users:** Stores citizen and administrator authentication and role data.
- **grievances:** The core table storing complaint details, current status, and image files.
- **complaint_locations:** Stores exact GPS coordinates for tracking merged/duplicate reports.
- **chat_logs:** Stores conversation history between the AI bot and citizens.
- **feedback:** Stores system ratings (1-5 stars) submitted by users.



Table Relationships & Cardinality

1. Users → Grievances [1:N]

Foreign Key: user_id

Details: One citizen can submit multiple complaints; each complaint is securely linked to one citizen.

2. Grievances → Complaint Locations [1:N]

Foreign Key: complaint_id

Details: One master grievance links to multiple GPS pins (powers duplicate detection in the same area).

3. Users → Complaint Locations [1:N]

• **Foreign Key:** user_id

• **Details:** One citizen can log multiple map pins, tracking exactly who reported which coordinate.

4. Users → Chat Logs [1:N]

• **Foreign Key:** user_id

• **Details:** One citizen can have many AI chat interactions; all conversation history is tied to their account.



SQL Queries / CRUD Operations

1. CRUD Operations (Create, Read, Update, Delete)

- **INSERT (Create):** Used when a citizen reports an issue
`INSERT INTO grievances (user_id, title, category, latitude, longitude) VALUES (?, ?, ?, ?, ?);`
- **SELECT (Read) with Conditions:** Used to load the User Dashboard.
`SELECT * FROM grievances WHERE user_id = ? AND status = 'Pending';`
- **UPDATE:** Used by Admin to change complaint status.
`UPDATE grievances SET status = 'Resolved', resolved_at = CURRENT_TIMESTAMP WHERE id = ?;`
- **DELETE:** Used by Admin to remove invalid reports.
`DELETE FROM grievances WHERE id = ?;`

2. Joins & Aggregate Functions

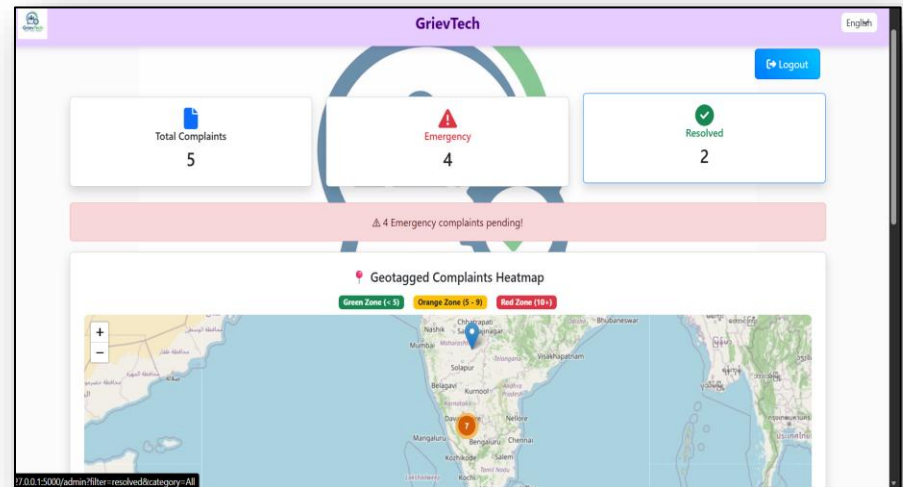
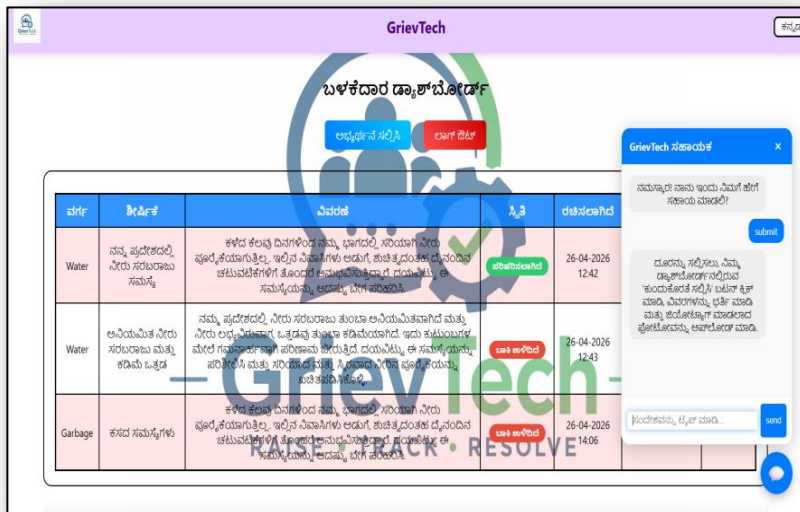
- **JOIN Operation:** Used to link citizen names to their complaints for the Admin view. `SELECT g.id, g.category, u.name FROM grievances g INNER JOIN users u ON g.user_id = u.id;`
- **Aggregate Function (COUNT, GROUP BY):** Used to generate the data for the Admin Bar Charts. `SELECT status, COUNT(*) as total_reports FROM grievances GROUP BY status;`



How SQL powers **Grievance System**

Feature	SQL Operation
 User Registration	INSERT INTO Users
 Login Authentication	SELECT FROM Users (verify credentials)
 Upload Grievance	INSERT INTO Grievances (store file & data)
 Duplicate Detection	SELECT (spatial math matching)
 Merge Duplicate Report	INSERT INTO Complaint_Locations
 View User Dashboard	SELECT FROM Grievances
 Admin View Heatmap	SELECT FROM Complaint_Locations
 Update Complaint Status	UPDATE Grievances (status = 'Resolved')
 Remove Invalid Report	DELETE FROM Grievances
 Chatbot Interaction (AI)	INSERT INTO Chat_Logs + AI response
 Submit System Feedback	INSERT INTO Feedback
 Generate Admin Charts	SELECT + GROUP BY (aggregate counts)

Frontend / Interface

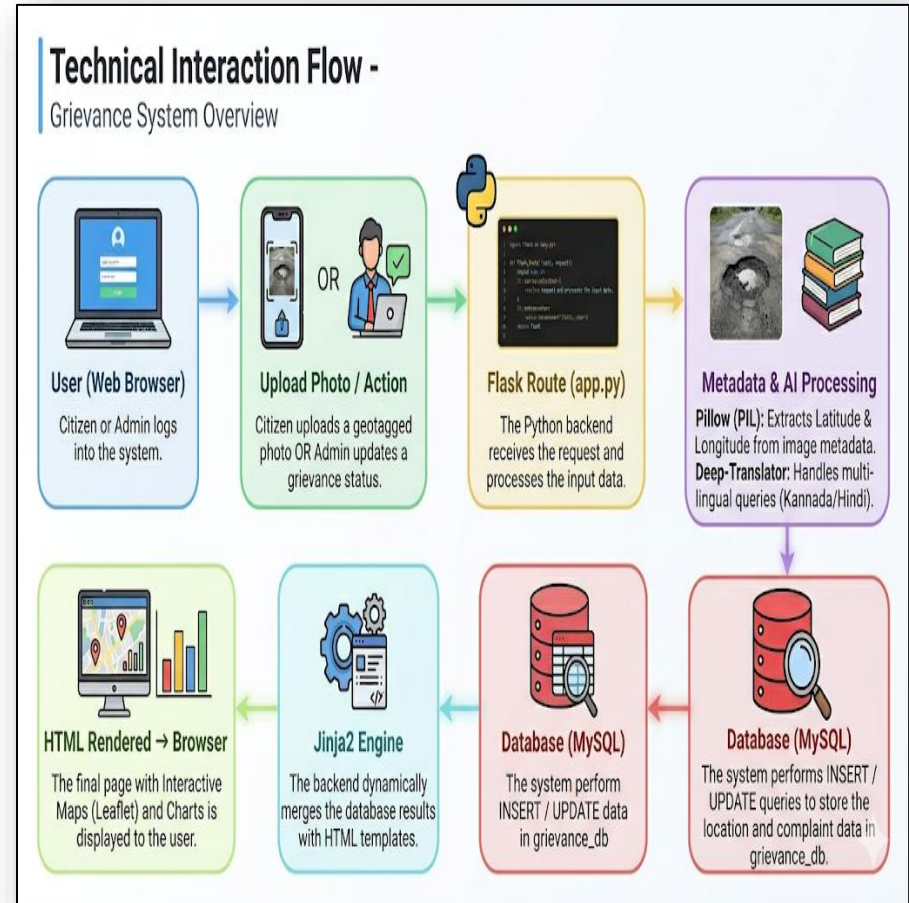




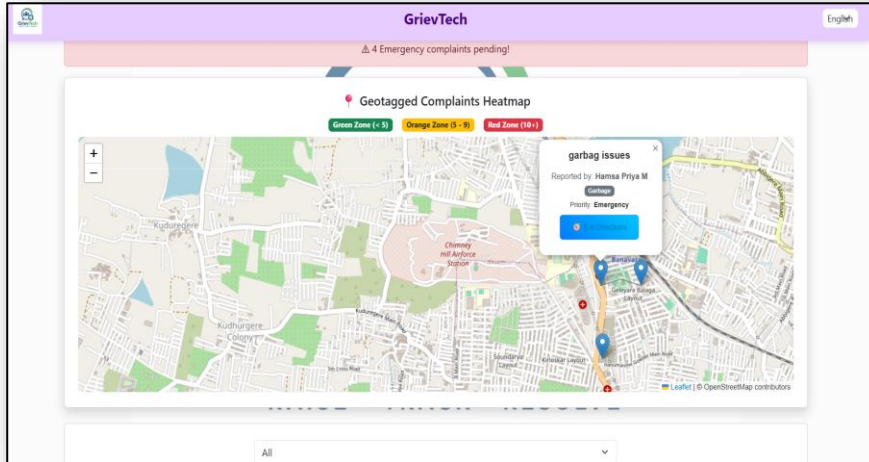
Frontend / Interface

Frontend Technologies Used:

- **HTML5 & CSS3:** For the core structure and custom styling (like the glassmorphism effects).
- **Bootstrap 5:** For the responsive layout, navigation bar, and interactive components like **Modals** and **Badges**.
- **JavaScript (ES6):** For the dynamic frontend logic and communicating with the backend without refreshing the page.
- **Leaflet.js:** The main library used to render the **Interactive Maps**.
- **Leaflet.markercluster:** A special plugin used to group multiple complaints into "Clusters" on the map (the Red/Orange/Green circles).
- **Chart.js:** Used to generate the **Bar Charts** and analytical graphs on the Admin Dashboard.
- **FontAwesome 6:** For all the professional icons (Trash can, Map pins, User icons, etc.).
- **Jinja2 (Templating Engine):** Technically part of Flask, but it is what allows you to inject Python data directly into your HTML pages using `{{ ... }}`.



Functionality and Output



ID	Category	Title	Description	Priority	reports	Status	File	Map Trace	Actions
9 (14,394, 74,533)	Electricity	electricity <1 reports	current is fluctuating and dim...	Emergency	2	In Progress	View	Trace	In Progress ▾ Update 
8 (13,072, 77,500)	Other	SNPSU Mess	In Saphagiri girls hostel we found a gutka packet...	Emergency	1	In Progress	View	Trace	In Progress ▾ Update 
7 (13,483, 77,504)	Electricity	Power Supply Issue in My Area <3 reports	There is frequent power interruption in our area. ...	Emergency	3	In Progress	View	Trace	In Progress ▾ Update 
5 (14,394, 74,533)	Garbage	garbage issues <1 reports	There is no proper water supply in our area for th...	Emergency	2	Pending	View	Trace	Pending ▾ Update 
3 (13,175, 77,48)	Water	Irregular Water Supply and Low Pressure	The water supply in our locality has been very in...	Normal	1	Pending	View	Trace	Pending ▾ Update 

Merged Reports for: electricity

#	Reported By	Problem Type	Trace	Update
1	Anusha D	Electricity	 	In Progress   
2	Hamsa Priya M	Electricity	 	In Progress   

ID	Category	Title	Description	Priority	reports	Status	File	Map Trace	Actions
9 (14,314, 74,323)	Electricity	electricity 	current is fluctuating in knd...	Emergency		In Progress			In Progress   
8 (13,672, 77,326)	Other	SNPSU Mess	In Sapthagiri girls hostel we found a gukka packet...	Emergency		In Progress			In Progress   
7 (13,687, 77,336)	Electricity	Power Supply Issue in My Area 	There is frequent power interruption in our area ...	Emergency		In Progress			In Progress   

[illegible]



Functionality and Output

User Action	SQL Runs (MySQL Database Query)	Result Shown
 User Registration	INSERT INTO users (name, email, password, role)	User registered successfully.
 Citizen Login	SELECT * FROM users WHERE email = ? AND password = ?	Citizen Dashboard loads with user data.
 Admin Login	SELECT * FROM users WHERE email = ? AND role = 'admin'	Admin Command Center loaded.
 Upload Grievance	INSERT INTO grievances (user_id, title, category, latitude, longitude)	Grievance & Geotag saved successfully.
 Duplicate Detection	SELECT * FROM grievances WHERE category = ? AND SQRT(...) < radius	System detects if report is new or duplicate.
 Merge Duplicate	INSERT INTO complaint_locations (complaint_id, latitude, longitude)	Map pin added to existing master report.
 Load Dashboards	SELECT * FROM grievances \$\$WHERE user_id = ?\$\$	Table populated; Heatmap & Charts generated.
 Trace Location	SELECT latitude, longitude FROM grievances WHERE id = ?	Interactive map centers directly on problem site.
 Update Status	UPDATE grievances SET status = ?, remarks = ? WHERE id = ?	Status updated; Citizen gets Email Alert.
 Delete Record	DELETE FROM grievances WHERE id = ?	Database and Server Files permanently deleted.
 AI Chatbot	INSERT INTO chat_logs (user_id, user_message, bot_response)	Bot replies dynamically in User's native language.
 Submit Feedback	INSERT INTO feedback (user_id, rating, message)	Feedback saved; Success message shown.



Innovation / Additional Features

1. **Multi-language Chatbot:** A chatbot that supports English, Kannada, and Hindi to help users ask questions and understand how to use the system
2. **Map-Based Complaint View:** Complaints are shown on a map so admin can easily identify problem areas
3. **Automatic Location Detection:** System can get location (latitude & longitude) from uploaded images, so users don't need to enter it manually
4. **Duplicate Complaint Detection:** Similar complaints are automatically grouped together to avoid repetition and increase priority
5. **Email Notifications:** Users receive updates when their complaint status changes
6. **Admin Dashboard:** Admin can view complaints, charts, and track exact locations with one click
7. **Security & Performance:** Secure login system and fast response for better user experience



Advantages and Limitations



Detailed Advantages



Precise Geotagging Accuracy

Automatically extracts exact GPS coordinates from image metadata, ensuring authorities receive 100% accurate location data without any manual user input errors.



Intelligent Deduplication System

Smart algorithms identify and merge multiple reports of the same issue in a specific vicinity, optimizing municipal resources and preventing administrative data clutter.



Real-time Progress Transparency

Empower citizens to monitor the entire lifecycle of their grievance—from initial submission to final resolution—via a transparent and interactive dashboard.



Strategic Resource Allocation

High-resolution heatmaps and analytical charts allow administrators to identify high-priority 'Red Zones' and deploy repair teams where they are needed most.



Multi-lingual Civic Accessibility

Full support for English, Kannada, and Hindi ensures that language barriers do not prevent any citizen from participating in community improvements.



Detailed Limitations & Disadvantages



Hardware Feature Dependency

The system's ability to extract accurate location data is entirely dependent on the user's smartphone having GPS functionality enabled at the moment of photo capture.



Constant Connectivity Requirements

Active internet access is mandatory for critical platform features such as real-time language translation, interactive map rendering, and automated notifications.



Administrative Oversight Necessity

While the reporting process is automated, the final 'Resolved' status still requires manual onsite verification by municipal authorities to ensure quality work.



Long-term Storage Scalability

Storing high-resolution photographic evidence for thousands of grievances can lead to significant server storage costs and database performance bottlenecks over time.



Conclusion

What the Project Does

- **GrievTech** is a web-based system built using **Python Flask and MySQL**.
- It allows citizens to report public grievances by simply uploading a **Geotagged photograph**.
- The system automatically extracts precise **GPS coordinates** (Latitude & Longitude) from image metadata.
- It provides a centralized platform to manage the complete workflow—from reporting to resolution.
- It handles core features like user authentication, location tracing, and **multi-lingual interaction**.

How It Solves the Problem

- **Eliminates manual location errors** by digitizing the entire reporting process.
- **Automatic Geotagging** ensures authorities find the exact problem site without vague descriptions.
- **Smart Deduplication logic** merges multiple reports of the same issue to prevent system clutter.
- **Real-time status tracking** keeps citizens informed and ensures administrative accountability.
- **Centralized Database (MySQL)** stores all spatial and grievance data for efficient retrieval.

Benefits User and Admin

- Citizens gain an easy, multi-lingual tool for reporting and tracking issues with precise GPS accuracy.
- Authorities benefit from a data-driven dashboard that eliminates duplicates and optimizes resource allocation.
- The system ensures 100% transparency and faster resolution of public grievances through automation.

GrievTech uses Geotagging and automation to provide a fast, transparent, and accurate way to manage public grievances. It improves civic engagement and municipal efficiency together.



Future Scope



FUTURE SCOPE

1



Mobile Application Launch

Developing dedicated Android and iOS apps to allow citizens to report issues on-the-go with real-time push notifications for status updates.

2



Advanced AI Integration

Implementing Computer Vision to automatically detect and categorize issue types (e.g., identifying a pothole vs. a water leak) from uploaded photos.

3



Cloud Database Migration

Moving to a cloud-based infrastructure (like AWS or Google Cloud) to ensure 24/7 global availability and high scalability for city-wide expansion.

4



Predictive Analytics Dashboard

Using Machine Learning to analyze historical data and predict issue-prone areas before problems even occur.

5



Automated Authority Routing

Integrating a smart dispatch system that automatically assigns grievances to the nearest available municipal field team based on their real-time GPS location.



References

Websites & Documentation

1. **Flask Official Documentation:** flask.palletsprojects.com
2. **MySQL Connector/Python Guide:** dev.mysql.com/doc/connector-python/
3. **W3Schools:** Python, SQL, and Bootstrap 5 Tutorials (w3schools.com)
4. **MDN Web Docs:** HTML5, CSS3, and JavaScript Reference (developer.mozilla.org)

Books & Textbooks

1. **"Database System Concepts"** by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan (Standard DBMS Textbook).
2. **"Flask Web Development"** by Miguel Grinberg (O'Reilly Media).



Thank You